

Benchmark: Evaluación Comparativa de Procesamiento Geoespacial

Anamaria Rodriguez Ramirez

2026-03-11

Contenido

0.1	Introducción	2
0.2	Ejecución de Códigos	2
0.2.1	Parte A - Ejecución en Jupyter (notebooks)	2
0.2.2	Parte B — Ejecución desde VSCode (terminal integrada)	4
0.2.3	Parte C — Ejecución desde el Termina de Windows (PowerShell)	7
0.3	Preguntas de análisis	9
0.3.1	Entorno de ejecución	9
0.3.2	Abstracción de la práctica	9
0.3.3	Julia y el costo de compilación (Warm-up)	9
0.3.4	Elección informada	10
0.4	Referencias	10

0.1 Introducción

El Benchmark, es una prueba de estandarización, que permite comparar el rendimiento del sistema, al realizar la ejecución de una misma tarea en distintos lenguajes de programación, en este caso en Python, R y Julia. El objetivo es realizar una operación aritmética a una única banda de raster (Banda 4 - Absorción de clorofila 665nm), para entender cuál es el costo computacional de cada lenguaje, y así poder tomar decisiones informadas sobre cuál utilizar en función de las necesidades del proyecto.

0.2 Ejecución de Códigos

0.2.1 Parte A - Ejecución en Jupyter (notebooks)

La ejecución de los scripts se realizó en un entorno de Jupyter Notebook, utilizando las bibliotecas específicas para cada lenguaje: `rasterio` para Python, `terra` y `stars` para R, y `Raster.jl` para Julia. Cada script fue diseñado para realizar la misma operación aritmética sobre la banda seleccionada del raster, asegurando que las condiciones de prueba fueran equivalentes.

Se observó que en Notebooks, el lenguaje con menor tiempo fue el de Python con la librería `rasterio`, con un tiempo de 12.952 seg, seguida de la librería Terra y Stars de R, para finalmente terminar con el lenguaje de Julia, con un tiempo de 56.159 seg. Lo anterior se observaba mejor en la siguiente gráfica

```

# Código en R
library(knitr)
library(kableExtra)

#Resultados de los lenguajes
t_python <- 12.952 #seg
t_julia <- 56.159 #seg
t_terra <- 25.119 #seg
t_stars <- 39.412 #seg

# Creamos el dataframe con los datos capturados
resultados <- data.frame(
  Motor = c("R: terra", "R: stars", "Python: rasterio", "Julia: Rasters.jl"),
  Lenguaje = c("R (C++)", "R", "Python (C++/NumPy)", "Julia (Nativo)"),
  Paralelismo = c("Monohilo", "Monohilo", "SIMD (Vectorizado)", "Multihilo (12 hilos)"),
  Tiempo_Seg = c(t_terra, t_stars, t_python, t_julia)
)
resultados


```

	Motor	Lenguaje	Paralelismo	Tiempo_Seg
1	R: terra	R (C++)	Monohilo	25.119
2	R: stars	R	Monohilo	39.412
3	Python: rasterio	Python (C++/NumPy)	SIMD (Vectorizado)	12.952
4	Julia: Rasters.jl	Julia (Nativo)	Multihilo (12 hilos)	56.159

```

# Cálculo de eficiencia: ¿Cuántas veces es más rápido que el más lento?
max_t <- max(resultados$Tiempo_Seg, na.rm = TRUE)
resultados$X_mas_rapido <- round(max_t / resultados$Tiempo_Seg, 2)

# Formateo elegante para el HTML
kable(resultados,
  digits = 3,
  caption = "Duelo de Titanes: Procesamiento de 1GB Sentinel-2 en JupyterLab Notebooks",
  col.names = c("Motor", "Lenguaje", "Paralelismo", "Tiempo (s)", "Eficiencia (X)")) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = F) %>%
  row_spec(which.min(resultados$Tiempo_Seg), bold = T, color = "white", background = "#2c3e50")

```

Table 1: Duelo de Titanes: Procesamiento de 1GB Sentinel-2 en JupyterLab Notebooks

Motor	Lenguaje	Paralelismo	Tiempo (s)	Eficiencia (X)
R: terra	R (C++)	Monohilo	25.119	2.24
R: stars	R	Monohilo	39.412	1.42
Python: rasterio	Python (C++/NumPy)	SIMD (Vectorizado)	12.952	4.34
Julia: Rasters.jl	Julia (Nativo)	Multihilo (12 hilos)	56.159	1.00

Asímismo, se observó con respecto a la generación de imágenes que el único lenguaje que presento problemas desde el Jupyter lab fue Julia, el cual no pudo generar la imagen, mientras que tanto Python como R si pudieron generar la imagen sin problemas.

En las siguientes imágenes se pueden observar los resultados en la interfaz del JupyterLab:

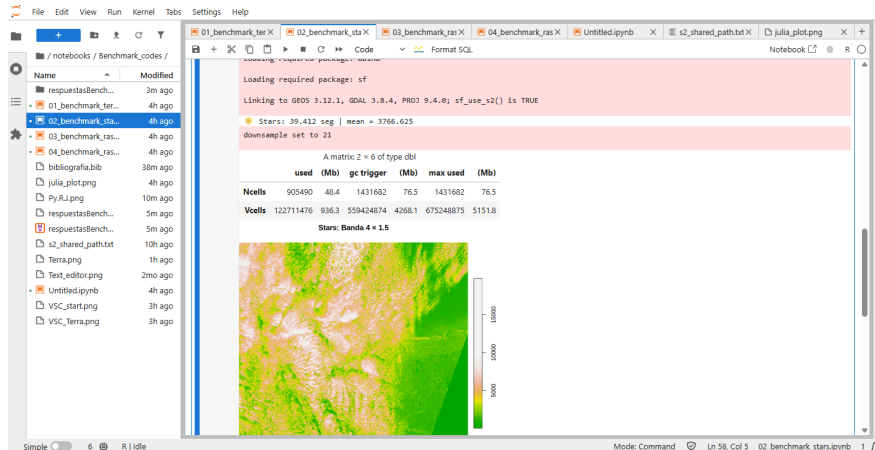


Figure 1: Cálculo aritmético de Banda 4 por un escalar en Lenguaje R con libreria start

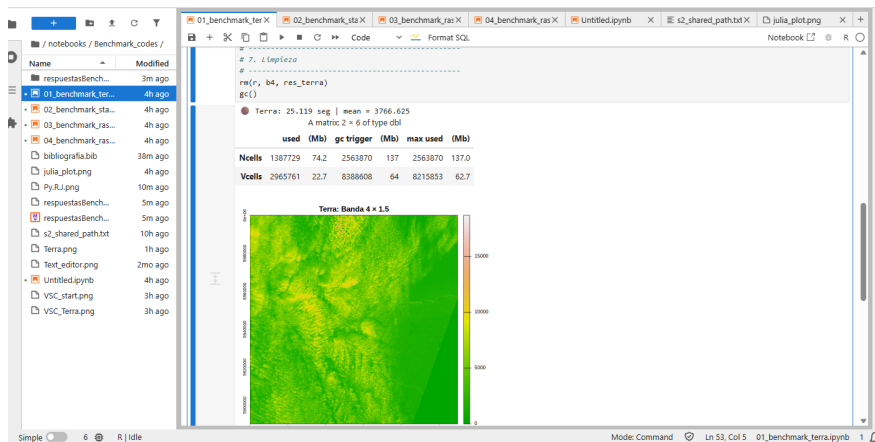


Figure 2: Cálculo aritmético de Banda 4 por un escalar en Lenguaje R con libreria terra

0.2.2 Parte B — Ejecución desde VSCode (terminal integrada)

Para VSCode, se crearon además de los archivos solicitados, uno para la creación del Path y otro con los resultados resumidos. Para este, se obtuvieron los siguientes resultados:

```

#Resultados de los lenguajes
library(knitr)
library(kableExtra)

#Resultados de los lenguajes

```

```

t_python <- 15.439 #seg
t_julia <- 56.585 #seg
t_terra <- 57.364 #seg
t_stars <- 1.438 #seg

# Creamos el dataframe con los datos capturados
resultados <- data.frame(
  Motor = c("R: terra", "R: stars", "Python: rasterio", "Julia: Rasters.jl"),
  Lenguaje = c("R (C++)", "R", "Python (C++/NumPy)", "Julia (Nativo)"),
  Paralelismo = c("Monohilo", "Monohilo", "SIMD (Vectorizado)", "Multihilo (12 hilos)"),
  Tiempo_Seg = c(t_terra, t_stars, t_python, t_julia)
)
resultados

```

	Motor	Lenguaje	Paralelismo	Tiempo_Seg
1	R: terra	R (C++)	Monohilo	57.364
2	R: stars	R	Monohilo	1.438
3	Python: rasterio	Python (C++/NumPy)	SIMD (Vectorizado)	15.439
4	Julia: Rasters.jl	Julia (Nativo)	Multihilo (12 hilos)	56.585

```

# Cálculo de eficiencia: ¿Cuántas veces es más rápido que el más lento?
max_t <- max(resultados$Tiempo_Seg, na.rm = TRUE)
resultados$X_mas_rapido <- round(max_t / resultados$Tiempo_Seg, 2)

# Formateo elegante para el HTML
kable(resultados,
  digits = 3,
  caption = "Duelo de Titanes: Procesamiento de 1GB Sentinel-2 en Visual Studio Code",
  col.names = c("Motor", "Lenguaje", "Paralelismo", "Tiempo (s)", "Eficiencia (X)")) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = F) %>%
  row_spec(which.min(resultados$Tiempo_Seg), bold = T, color = "white", background = "#2c3e50")

```

Table 2: Duelo de Titanes: Procesamiento de 1GB Sentinel-2 en Visual Studio Code

Motor	Lenguaje	Paralelismo	Tiempo (s)	Eficiencia (X)
R: terra	R (C++)	Monohilo	57.364	1.00
R: stars	R	Monohilo	1.438	39.89
Python: rasterio	Python (C++/NumPy)	SIMD (Vectorizado)	15.439	3.72
Julia: Rasters.jl	Julia (Nativo)	Multihilo (12 hilos)	56.585	1.01

Observandose qué en ejecución el más rápido fue el lenguaje de R con la librería Stars, seguido de Python con Rasterio, luego R con Terra y finalmente Julia con Rasters.jl.

La imagen generada por el lenguaje de Julia, que no pudo ser visualizada en JupyterLab, fue la siguiente:

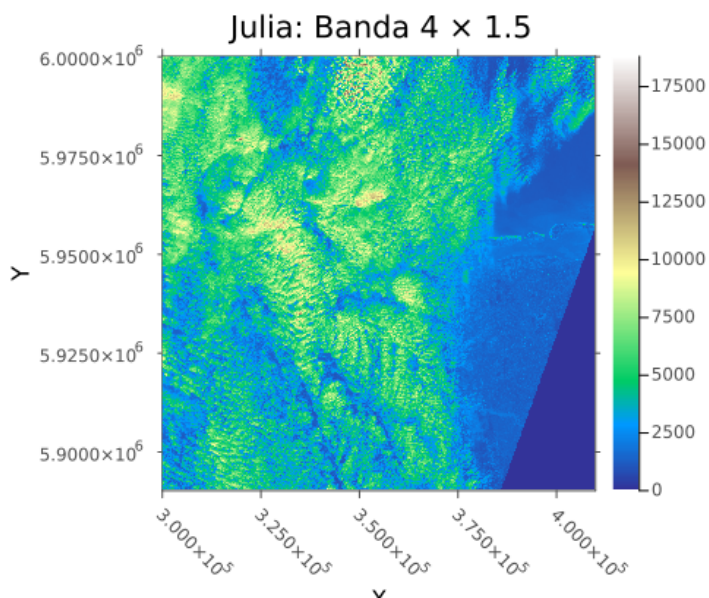


Figure 3: Cálculo aritmético de Banda 4 por un escalor

Asímismo, la ejecución de códigos en VSCoDe se vió de la siguiente forma:

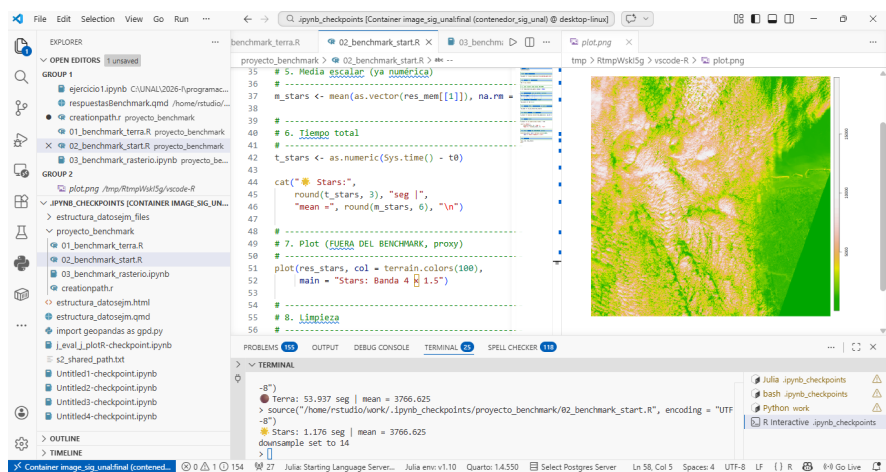


Figure 4: Cálculo aritmético de Banda 4 por un escalor en Lenguaje R con libreria start

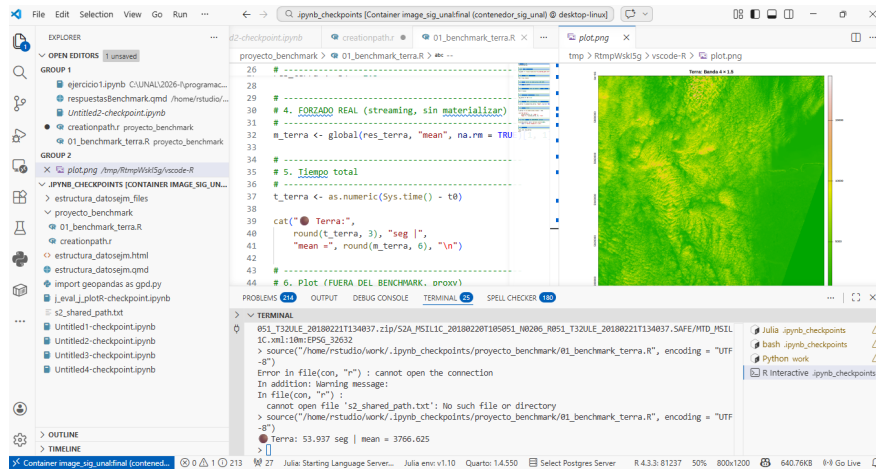


Figure 5: Cálculo aritmético de Banda 4 por un escalar en Lenguaje R con libreria terra

0.2.3 Parte C — Ejecución desde el Termina de Windows (PowerShell)

A diferencia de VSCode y los Notebooks, no se puede realizar visualización de las imágenes, sin embargo para obtener cálculos y el valor de los tiempos, la terminal es mucho más eficiente, dado que el mayor tiempo fue el empleado por Terra de R, con 45.412. Los valores obtenidos se ven en la siguiente tabla:

```
#Resultados de los lenguajes
library(knitr)
library(kableExtra)

#Resultados de los lenguajes
t_python <- 7.929 #seg
t_julia <- 25.566 #seg
t_terra <- 45.412 #seg
t_stars <- 31.207 #seg

# Creamos el dataframe con los datos capturados
resultados <- data.frame(
  Motor = c("R: terra", "R: stars", "Python: rasterio", "Julia: Rasters.jl"),
  Lenguaje = c("R (C++)", "R", "Python (C++/NumPy)", "Julia (Nativo)"),
  Paralelismo = c("Monohilo", "Monohilo", "SIMD (Vectorizado)", "Multihilo (12 hilos)"),
  Tiempo_Seg = c(t_terra, t_stars, t_python, t_julia)
)
resultados
```

	Motor	Lenguaje	Paralelismo	Tiempo_Seg
1	R: terra	R (C++)	Monohilo	45.412
2	R: stars	R	Monohilo	31.207
3	Python: rasterio	Python (C++/NumPy)	SIMD (Vectorizado)	7.929
4	Julia: Rasters.jl	Julia (Nativo)	Multihilo (12 hilos)	25.566

```
# Cálculo de eficiencia: ¿Cuántas veces es más rápido que el más lento?
max_t <- max(resultados$Tiempo_Seg, na.rm = TRUE)
resultados$X_mas_rapido <- round(max_t / resultados$Tiempo_Seg, 2)

# Formateo elegante para el HTML
kable(resultados,
      digits = 3,
      caption = "Duelo de Titanes: Procesamiento de 1GB Sentinel-2 en la Terminal",
      col.names = c("Motor", "Lenguaje", "Paralelismo", "Tiempo (s)", "Eficiencia (X)")) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
               full_width = F) %>%
  row_spec(which.min(resultados$Tiempo_Seg), bold = T, color = "white", background = "#2c3e50")
```

Table 3: Duelo de Titanes: Procesamiento de 1GB Sentinel-2 en la Terminal

Motor	Lenguaje	Paralelismo	Tiempo (s)	Eficiencia (X)
R: terra	R (C++)	Monohilo	45.412	1.00
R: stars	R	Monohilo	31.207	1.46
Python: rasterio	Python (C++/NumPy)	SIMD (Vectorizado)	7.929	5.73
Julia: Rasters.jl	Julia (Nativo)	Multihilo (12 hilos)	25.566	1.78

En general, se observa que el lenguaje de Python con Rasterio es el más eficiente en términos de tiempo de ejecución, aplicado desde la terminal y JupyterLab. Mientras que el lenguaje de R con la librería Stars es el más eficiente en términos de tiempo de ejecución, aplicado desde Visual Studio Code. Por otro lado, el lenguaje de Julia con Rasters.jl es el menos eficiente en términos de tiempo de ejecución, aplicado desde JupyterLab y Visual Studio Code. Sin embargo, es importante destacar que los resultados pueden variar dependiendo del entorno de ejecución y la configuración del sistema.

La ejecución del código se visualizó en la terminal de la siguiente forma:

```

* Stars: 31.207 seg | mean = 3766.625
downsample set to 28
used (Mb) gc trigger (Mb) max used (Mb)
Ncells 665708 35.6 1257296 67.2 1814414 54.2
Vcells 121700346 928.5 558821095 4263.5 674343764 5144.9
PS C:\UNAL\2026-I\programacion_SIG\ProgramacionSIG2026> docker exec contenedor_sig_unal bash -c "cd /home/rstudio/work/p
royecto_benchmark && Rscript 03_benchmark_rasterio.py"
Error: unexpected symbol in "import rasterio"
Execution halted
PS C:\UNAL\2026-I\programacion_SIG\ProgramacionSIG2026> docker exec contenedor_sig_unal bash -c "cd /home/rstudio/work/p
royecto_benchmark && python3 03_benchmark_rasterio.py"
Python: 7.929 seg | mean = 3766.624630
PS C:\UNAL\2026-I\programacion_SIG\ProgramacionSIG2026> docker exec contenedor_sig_unal bash -c "cd /home/rstudio/work/p
royecto_benchmark && julia 04_benchmark_rasters.jl"
Julia: 25.566 seg | mean = 3766.62463

```

Figure 6: Cálculo aritmético de Banda 4 por un escalor en Lenguaje R, Python y Julia


```

Attaching package: 'terra'

The following object is masked from 'package:grid':

    depth

● Terra: 45.412 seg | mean = 3766.625
      used (Mb) gc trigger (Mb) max used (Mb)
Ncells 1157360 61.9   2206688 117.9  2206688 117.9
Vcells 1529079 11.7   8388608  64.0  8125124  62.0
PS C:\UNAL\2026-I\programacion_SIG\ProgramacionSIG2026>

```

Figure 7: Cálculo aritmético de Banda 4 por un escalar en Lenguaje R con libreria terra en la Terminal

0.3 Preguntas de análisis

0.3.1 Entorno de ejecución

Las principales diferencias en los tiempos de ejecución entre los entornos dependen del flujo de trabajo, no solo del lenguaje de programación empleado, sino también el entorno sobre el cuál se ejecuta.

Para el caso de la terminal, es el entorno más siliar al sistema operativo, por lo que no requiere de compilación kernels interactivos ni interfases, generando que el flujo de trabajo sea más directo, lo que se traduce en tiempos de ejecución más cortos.

En cuanto al JupyterLab, necesita además de compilarse, generar comunicación con el navegador, generando un aumento en los tiempos. Finalmente, para Visual Studio Code, aunque es un entorno de desarrollo integrado, también requiere de una comunicación con el sistema operativo y la generación de una interfaz gráfica, lo que puede generar tiempos de ejecución más largos en comparación con la terminal.

0.3.2 Abstracción de la práctica

El costo de las abtracciones es más visible en la libreria de **stars** (R), debido a la manera en la que funciona la libreria, al estar diseñada para datos espaciales multidimensionales, requiere de varias capas, generando un mayor costo administrativo. No trabaja con datos menos complejos como las matrices en **terra**, sino que trabaja con objetos más complejos (cmo bandas, tiempos, variables), lo que puede generar un mayor costo computacional, por lo que presenta tiempos mayores de ejecución con respecto las otras liberias. En contraste, motores como terra o rasterio trabajan más cerca de bibliotecas optimizadas en C/C++ o NumPy, reduciendo el impacto de dichas capas intermedias.

0.3.3 Julia y el costo de compilación (Warm-up)

¿El efecto del warm-up de Julia se notó más en algún entorno específico? - Expliquen brevemente por qué

Entendiendo el efecto del **warm-up**, como el tiempo adicional para compilación de funciones en su primera ejecución, debido al modelo de compilación que maneja el lenguaje Julia, donde evalúa al final, se muestran tiempos muy altos en los entornos de JupyterLab y VSCode, en comparación con la terminal, donde el tiempo fue de 25s. Esto se debe a los procesos de kernel y compilación que se generan en los entornos de JupyterLab y VSCode, lo que puede aumentar significativamente el tiempo de ejecución inicial, mientras que en la terminal, el proceso es más directo y no requiere de la misma cantidad de recursos para la compilación, resultando en tiempos de ejecución más cortos.

0.3.4 Elección informada

Dependiendo del tipo de proceso que requiera realizar, si elegiría el entorno de ejecución y asimismo el lenguaje. Para cálculos rápidos a menores tiempos, elegiría a Python por su velocidad en resultados y rapidez de ejecución, además de su capacidad de flujos de datos y automatización siendo este un factor clave para la toma de decisiones rápidas.

El anterior ejercicio fue realizado gracias a la guía de la clase de Rodríguez-Avellaneda (2026), de los apartados **Benchmark geoespacial: cómo leer los resultados** y **Evaluación Comparativa de Procesamiento Geoespacial**, además de los resultados obtenidos en la ejecución de los códigos en los diferentes entornos.

0.4 Referencias

Rodríguez-Avellaneda, A. H. (2026). Programación en SIG: R, Python y Julia [Material de curso]. Maestría en Geomática, Universidad Nacional de Colombia. Recuperado de <https://github.com/alexyshr/ProgramacionSIG2026>